

Domain-Disguised Coding Problems

Real interview questions dressed in a company's business domain — and how to "de-story" them

Target level: Mid-level software engineering interviews at companies such as Booking.com, Airbnb, Microsoft, Agoda, UiPath, and organizations of similar range.

What this volume is

Interviewers at these companies rarely say "solve *Two Sum*." They say "a guest is looking for two dates whose prices add up to their budget...". Same algorithm, wrapped in a business story. This round tests one extra skill on top of DSA: **can you strip the story and see the pattern underneath?**

Every problem here is presented the way an interviewer poses it, then "**de-storied**" into the plain algorithmic problem, then solved (brute force → optimized Java) with complexity.

Evidence tags — read this first

This volume is built from a **verified research pass**. Each problem carries a tag showing how strongly it's tied to a real interview:

- **[REPORTED — Company, date]** — a dated, first-person interview report (LeetCode Discuss). The strongest evidence: a real candidate says they were asked this.
- **[COMPANY-TAGGED — Company]** — the problem appears in that company's LeetCode company-tag frequency data. Strong signal that the *problem* is asked there, though the "story" wrapper is our illustration, not a quoted transcript.
- **[CANONICAL]** — the standard, widely-used analog for a class of questions (e.g., the textbook "network bridges" problem).
- **[CONCEPTUAL — not a reported question]** — included for learning value, with an explicit warning that our research found **no evidence** any of these companies actually asked it.

Why the tags matter: if you claim in an interview "I know Airbnb asks X", it had better be true. This volume is honest about what's confirmed vs. inferred so you never over-claim.

The one skill this trains: de-storying

1. Ignore the nouns (hotels, guests, tasks) — they're decoration.
2. Find the INPUT (an array? a graph? intervals?) and the OUTPUT (a count? a min/max? a boolean? all combinations?).
3. Find the CONSTRAINT (sorted? within a budget? no overlaps?).
4. Match to a known pattern. The story almost never changes the pattern.

The problems in this volume

#	Story	Company	Pattern	Evidence
1	Property Booking Optimizer	Airbnb	subset / knapsack (backtracking)	REPORTED
2	Spend the Whole Budget	Booking.com	backtracking enumeration	REPORTED
3	Hotel Inventory Aggregation	Agoda	systems design	REPORTED
4	Reconstruct the Itinerary	Booking.com	Eulerian path (Hierholzer)	COMPANY-TAGGED
5	Recently-Used Cache	Booking.com	design (hashmap + DLL)	COMPANY-TAGGED
6	Peak Price in Each Window	Booking.com	monotonic deque	COMPANY-TAGGED
7	Escape the Warehouse Grid	UiPath	bounded BFS	COMPANY-TAGGED
8	Biggest Connected Zone	UiPath	union-find / DFS	COMPANY-TAGGED
9	Consistent Rules Check	UiPath	union-find	COMPANY-TAGGED
10	Prune the Bot Task Tree	UiPath	graph leaf-trim / topo	COMPANY-TAGGED
11	Critical Network Links	(networking)	bridges (Tarjan's)	CANONICAL
A	IP Routing via Trie	(networking)	trie / longest-prefix	CONCEPTUAL

1. Property Booking Optimizer

Evidence: [REPORTED — Airbnb, Senior SWE technical screen, May 5 2025 (LeetCode Discuss #6717917)]

How the interviewer poses it. "We have a list of property listings. Each listing has an `id`, a `neighborhood`, and a `capacity` (how many guests it sleeps). A group of `groupSize` guests wants to stay in a specific neighborhood. Pick a **combination of listings** in that neighborhood whose **total capacity is at least the group size**, while wasting as little space as possible — i.e., **minimize the excess** (total capacity – group size). If there's a tie, prefer the combination that uses **fewer properties**."

De-story it

Strip the travel nouns:

```
nouns to ignore: "listing", "neighborhood", "guests"
INPUT    : a list of positive integers (the capacities, after filtering
           to the requested neighborhood)
OUTPUT   : the subset whose sum is >= target and whose (sum – target)
           is smallest; tie-break on smaller subset size
CONSTRAINT: "at least" the target, minimize the overshoot
```

That is a **subset-sum / bounded-knapsack** problem: choose a subset of items to hit a target as tightly as possible. Each listing is used at most once → 0/1 choice per item → **classic "include / exclude" backtracking** (or subset-sum DP).

Worked example

```
listings in the target neighborhood (capacities): [4, 3, 2, 5]
groupSize = 6

candidate subsets that reach >= 6:
[4,3]   = 7  -> excess 1   (2 properties)
[4,2]   = 6  -> excess 0   (2 properties)  <-- best
[5,2]   = 7  -> excess 1
[4,3,2] = 9  -> excess 3
Answer: [4,2] (excess 0, and no single listing reaches 6)
```

Intuition (diagram)

At each listing you make one binary decision: **take it or skip it**. That builds a binary decision tree; each leaf is one possible combination whose total we test against the target.

```

                start (sum=0)
            take 4 /      \ skip 4
            (sum=4)        (sum=0)
        take 3 / \ skip3   take 3 / \ skip 3
        (7)*   (4)       (3)       (0)
        ...     ...     ...     ...
    * every node with sum>=6 is a candidate; keep the min-excess one

```

Brute force

Enumerate **all** 2^n subsets, keep the best that reaches the target.

```

public List<Integer> optimizeBrute(int[] cap, int groupSize) {
    int n = cap.length;
    List<Integer> best = null;
    int bestExcess = Integer.MAX_VALUE;
    for (int mask = 0; mask < (1 << n); mask++) {    // every subset
        int sum = 0;
        List<Integer> pick = new ArrayList<>();
        for (int i = 0; i < n; i++) {
            if ((mask & (1 << i)) != 0) { sum += cap[i]; pick.add(cap[i]); }
        }
        if (sum >= groupSize) {
            int excess = sum - groupSize;
            if (excess < bestExcess ||
                (excess == bestExcess && best != null && pick.size() < best.size())) {
                bestExcess = excess;
                best = pick;
            }
        }
    }
    return best;
}

```

Optimized (backtracking with pruning)

Same decision tree, but **prune** branches that can't beat the best excess found so far, and stop early on a perfect (excess 0) fit.

```

private List<Integer> best;
private int bestExcess;

public List<Integer> optimize(int[] cap, int groupSize) {
    best = null;
    bestExcess = Integer.MAX_VALUE;
    Arrays.sort(cap); // helps pruning + finds tight fits sooner
    backtrack(cap, 0, groupSize, 0, new ArrayList<>());
    return best;
}

private void backtrack(int[] cap, int i, int target, int sum, List<Integer> cur) {
    if (sum >= target) { // reached the goal on this path
        int excess = sum - target;
        if (excess < bestExcess ||
            (excess == bestExcess && (best == null || cur.size() < best.size()))) {
            bestExcess = excess;
            best = new ArrayList<>(cur);
        }
        return; // adding more can only increase excess
    }
    if (i == cap.length || bestExcess == 0) return; // no items left, or perfect fit found

    cur.add(cap[i]); // choice: TAKE listing i
    backtrack(cap, i + 1, target, sum + cap[i], cur);
    cur.remove(cur.size() - 1); // undo

    backtrack(cap, i + 1, target, sum, cur); // choice: SKIP listing i
}

```

Version	Time	Space
Brute force (all subsets)	$O(2^n \cdot n)$	$O(n)$
Backtracking + pruning	$O(2^n)$ worst case, far less in practice	$O(n)$ recursion

If the interviewer pushes for polynomial: when capacities are small integers, this becomes a **subset-sum DP** — $dp[s]$ = min items to reach capacity sum s — giving $O(n \cdot \text{maxSum})$. Mention it as the scale-up answer.

What to say

"Ignoring the travel framing, this is subset-sum: pick a subset of capacities that meets the group size with minimal overshoot. Each listing is a take-or-skip 0/1 decision, so I'll backtrack over the decision tree, prune branches that can't beat my best excess, and stop on a perfect fit. If capacities are bounded I can switch to a subset-sum DP for polynomial time."

2. Spend the Whole Budget

Evidence: [REPORTED — Booking.com, SDE II Bengaluru, Jun 7 2025 (LeetCode Discuss #6819299)]

How the interviewer poses it. "You're planning a multi-city trip with a fixed `budget`. Each city — say Paris, LA, Amsterdam — has a **per-day price** to stay there. You can spend any whole number of days in a city (including zero, and you're free to revisit the idea of 'more days' in the same city). Find **all the different ways** to allocate your days across cities so that the **total cost is exactly the budget**, with no money left over."

De-story it

Strip the travel nouns:

```
nouns to ignore: "city", "trip", "days", "Paris/LA/Amsterdam"
INPUT    : a list of positive integers (the per-day prices) and a
           target integer (the budget)
OUTPUT   : all combinations of prices (a price may repeat, since you
           can spend multiple days in the same city) whose sum
           equals the target exactly
CONSTRAINT: sum must equal the target EXACTLY (not >=, not <=);
           each "item" (city) may be reused any number of times
```

That is **combination sum with unlimited reuse** — the same shape as LeetCode 39 (Combination Sum). Because a candidate can be picked again and again (more days in the same city), this is **enumeration via backtracking/DFS**, not classic DP: the interviewer wants the actual list of combinations, not just a count or an optimum, so we must explore and collect every valid path through the decision tree.

Worked example

```
prices (per-day costs): [100, 200, 300]    (Paris=100, LA=200, Amsterdam=300)
budget = 500

valid combinations (order-independent, each price reusable):
[100,100,100,100,100]    -> 5 days in Paris
[100,100,300]            -> 2 days Paris + 1 day Amsterdam
[200,300]                -> 1 day LA + 1 day Amsterdam
[100,200,200]            -> 1 day Paris + 2 days LA
Answer: those 4 combinations (as multisets of prices)
```

Intuition (diagram)

At each step, pick a price at or after the current index, or move on to the next price. Subtract it from the remaining budget; stop when remaining hits 0 (record it) or goes negative (prune).

```

        remaining = 500, start = 0
    pick 100 /          \ skip 100, start = 1
    remaining=400, start=0    remaining=500, start=1
    pick 100 / \ skip100    pick 200 / \ skip 200
    ...          ...      ...          ...
    leaf where remaining == 0 -> record current combo as an answer
    leaf where remaining < 0 -> prune, backtrack

```

Brute force

Try every price at every step without a start index — this finds all combinations but also revisits the same multiset in different orders (e.g. `[100,200]` and `[200,100]`), so it needs a dedupe pass (typically via sorting + a "seen" set) to match the required output.

```

public List<List<Integer>> spendBrute(int[] prices, int budget) {
    List<List<Integer>> raw = new ArrayList<>();
    dfsBrute(prices, budget, new ArrayList<>(), raw);

    // dedupe combinations that are the same multiset in different orders
    Set<List<Integer>> unique = new LinkedHashSet<>();
    for (List<Integer> combo : raw) {
        List<Integer> sorted = new ArrayList<>(combo);
        Collections.sort(sorted);
        unique.add(sorted);
    }
    return new ArrayList<>(unique);
}

private void dfsBrute(int[] prices, int remaining, List<Integer> cur,
    List<List<Integer>> out) {
    if (remaining == 0) {
        out.add(new ArrayList<>(cur));
        return;
    }
    if (remaining < 0) return;

    for (int p : prices) {          // no start index -> every order is explored
        cur.add(p);
        dfsBrute(prices, remaining - p, cur, out);
        cur.remove(cur.size() - 1);
    }
}

```

Optimized (backtracking with a start index)

Only ever pick prices at index `>= start`, so within one path prices only move forward. That guarantees each multiset is built exactly once — no dedupe step needed — and we prune the moment `remaining` goes negative.

```

public List<List<Integer>> spend(int[] prices, int budget) {
    List<List<Integer>> result = new ArrayList<>();
    Arrays.sort(prices); // optional: enables early-exit pruning
    backtrack(prices, budget, 0, new ArrayList<>(), result);
    return result;
}

private void backtrack(int[] prices, int remaining, int start,
                      List<Integer> cur, List<List<Integer>> result) {
    if (remaining == 0) {
        result.add(new ArrayList<>(cur));
        return;
    }
    if (remaining < 0) return;

    for (int i = start; i < prices.length; i++) {
        if (prices[i] > remaining) break; // sorted -> nothing further can fit either
        cur.add(prices[i]);
        backtrack(prices, remaining - prices[i], i, cur, result); // i, not i+1: reuse allow
        cur.remove(cur.size() - 1); // undo
    }
}

```

Version	Time	Space
Brute force (all orderings + dedupe)	$O(\text{budget}^n)$ explored, plus dedupe overhead	$O(\text{budget}^n)$ for raw results
Backtracking (start index, no reuse of order)	exponential in worst case — this is enumeration, so the output itself can be exponentially large	$O(\text{budget} / \text{minPrice})$ recursion depth

Be honest about the bound: this problem is fundamentally about listing every valid combination, so there's no polynomial-time algorithm that "solves" it faster — the best we can do is avoid *wasted* work (duplicate orderings, dead branches past the budget). If the interviewer only wants the **count** of ways, that degrades to an unbounded-knapsack **DP** ($\text{dp}[b] = \text{sum of dp}[b - \text{price}]$ over prices), which is polynomial — mention it as the follow-up.

What to say

"Once I strip the cities away, this is Combination Sum: prices are candidates I can reuse (more days in the same city), and I need every combination that sums exactly to the budget. I'll backtrack with a start index so I only build each multiset once, in sorted order, and prune as soon as the remaining budget goes negative or a price exceeds it. This is enumeration, so it's inherently exponential — if they only wanted the *count* of ways, I'd switch to an unbounded-knapsack DP instead."

3. Hotel Inventory Aggregation

Evidence: [REPORTED — Agoda, Senior SWE (8 YOE) virtual onsite, Jun 18 2025
(LeetCode Discuss #6859065)]

How the interviewer poses it. "Design the backend for a hotel search page. When a guest searches a city and date range, we need to show room availability and price that comes from our **own inventory database** as well as from **external partners** like MakeMyTrip and Booking.com. Walk me through how you'd build the service that gathers all of this and returns one merged result to the client."

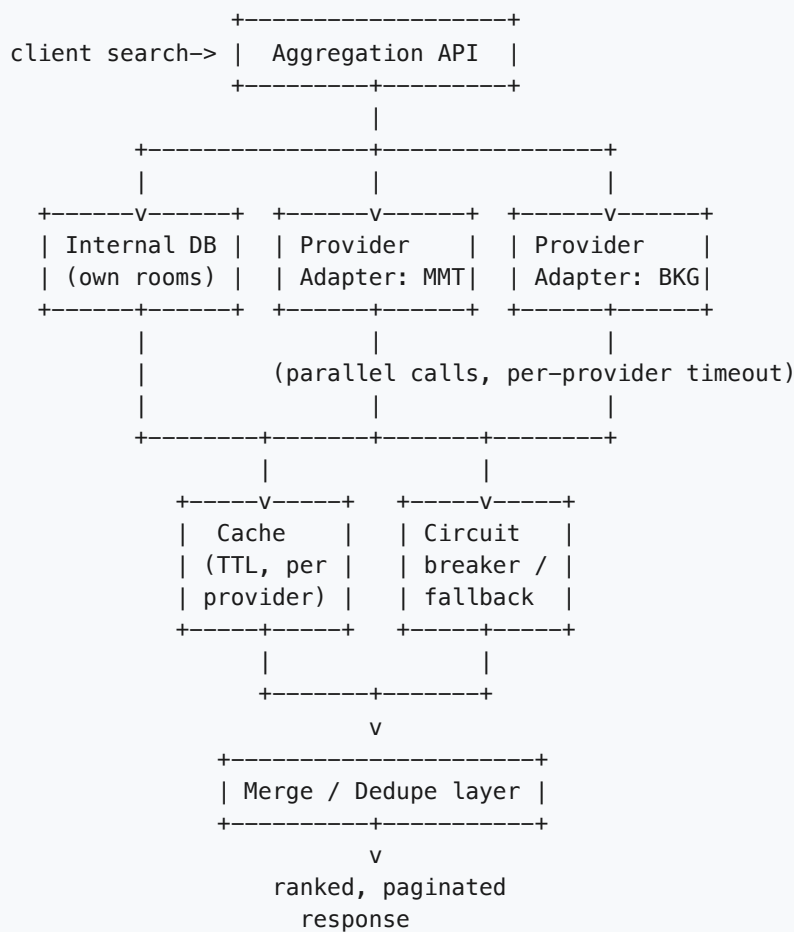
What they're really testing

This one showed up in a **platform/systems round**, not a coding round — there's no array to manipulate and no complexity to optimize, so don't reach for Java or a Big-O table here. It's a **systems-design / service-aggregation** problem: the interviewer wants to see how you compose an internal data source with several flaky, slow, third-party APIs into a single reliable response. This belongs squarely in the HLD skillset — if you've worked through the building blocks in the HLD guides (caching layers, parallel fan-out, timeout/circuit-breaker patterns, graceful degradation), you already have the vocabulary for this answer. The "hotel" framing is just a skin over "aggregate from N heterogeneous backends under a latency budget."

Requirements to clarify

- How many external providers do we integrate with today, and is that list expected to grow?
- What's the acceptable end-to-end latency for a search request (p95/p99)?
- Is partial data acceptable — i.e., can we return internal + 2-of-3 providers if one is slow?
- How fresh does external inventory need to be? Can we tolerate a few minutes of staleness?
- Do providers rate-limit us, and do we have per-provider SLAs or contracts to respect?
- Same room/hotel can appear from multiple sources — how do we identify it's the "same" room (shared hotel ID mapping, fuzzy match on name+address, etc.)?
- Read-heavy and bursty (flash sales, holiday search spikes) — do we need to scale reads independently of the write/ingestion path?

High-level design



An **aggregation service** sits in front of the internal inventory DB and a set of provider adapters (one per external partner: MakeMyTrip, Booking.com, ...). On a search request it queries the internal DB directly and **fans out in parallel** to each provider adapter, each guarded by its own timeout. Each adapter first checks a **cache** keyed on (city, dates, provider) before hitting the real API, since provider calls are the slowest and most rate-limited part of the pipeline. Results — internal + whatever providers answered in time — flow into a **merge/dedupe layer** that reconciles the same physical room/hotel appearing from multiple sources, then the combined, paginated list goes back to the client. If a provider is down or too slow, its circuit breaker trips and we degrade gracefully by simply omitting that source rather than blocking the whole response.

Key design points

- **Caching external results with TTL.** Provider calls are the highest-latency, most rate-limited hop, so cache per (city, date-range, provider) with a short TTL (minutes, not hours) — freshness matters for price/availability, but not to the second.
- **Parallel fan-out with per-call timeout.** Query internal DB and all provider adapters concurrently (not sequentially), each bounded by its own timeout so one slow provider doesn't stall the whole request; use whatever responded within the budget.
- **Circuit breaker / graceful degradation.** If a provider's error rate or latency crosses a threshold, trip its breaker and stop calling it for a cool-down window, returning results from the remaining

sources instead of failing the whole search.

- **Deduplication / merging.** The same hotel/room can appear from our own inventory and from one or more providers — merge on a stable key (mapped hotel ID) or fuzzy-match on name + address + room type, and decide a tie-break rule (e.g., prefer internal price if present, else lowest of the external quotes).
- **Rate limiting against providers.** Respect each partner's contracted call limits with a per-provider rate limiter/token bucket so a traffic spike on our side doesn't get us throttled or blacklisted by the partner.
- **Pagination.** The merged result set can be large; paginate (cursor or offset) after merge/rank so the client gets a stable, bounded page rather than the full fanned-out set.

What to say

"Strip the travel framing and this is a fan-out aggregation problem: one internal data source plus N external APIs that need to be combined into a single response under a latency budget. I'd query the internal DB and call each provider adapter in parallel, each with its own timeout, backed by a per-provider TTL cache to avoid hammering slow, rate-limited partners. A circuit breaker per provider lets me degrade gracefully — if MakeMyTrip is down, I still return internal + Booking.com results instead of failing the whole search. After fan-out, a merge/dedupe layer reconciles the same room appearing from multiple sources before paginating the final list back to the client."

4. Reconstruct the Itinerary

Evidence: [COMPANY-TAGGED — Booking.com (LeetCode 332, ~62.5% frequency)]

How the interviewer poses it. "You're given a list of airline tickets, each written as a [from, to] pair of airport codes. All the tickets belong to a single traveler who started their trip at JFK . Reconstruct the **order of the trip** using **every ticket exactly once**. If more than one valid itinerary exists, return the one that is **lexicographically smallest** when read as a sequence of airport codes."

De-story it

Strip the travel nouns:

```
nouns to ignore: "airline", "traveler", "airport", "trip"
INPUT   : a list of directed pairs [from, to] (edges in a graph)
OUTPUT  : a single ordering of all nodes, starting at "JFK", that uses
          every edge exactly once
CONSTRAINT: if multiple valid orderings exist, return the
            lexicographically smallest one
```

That is a **Eulerian path** problem: each ticket is a directed edge, and "use every ticket exactly once" is the textbook definition of a path that traverses every edge in the graph exactly once. The lexicographic tie-break means that at every node you should greedily try the smallest-lettered destination first — which is exactly what **Hierholzer's algorithm** gives you when the adjacency lists are kept sorted.

Worked example

```
tickets: [["MUC","LHR"], ["JFK","MUC"], ["SFO","SJC"], ["LHR","SFO"]]

adjacency (sorted):
JFK -> [MUC]
MUC -> [LHR]
LHR -> [SFO]
SFO -> [SJC]

walk from JFK, always taking the smallest unused edge:
JFK -> MUC -> LHR -> SFO -> SJC

Answer: ["JFK","MUC","LHR","SFO","SJC"] (all 4 tickets used exactly once)
```

Intuition (diagram)

```
JFK --> MUC --> LHR --> SFO --> SJC
each edge = one ticket; visit every edge exactly once
at a fork, always leave via the smallest airport code
```

Brute force

Backtrack over every possible ordering of tickets starting at `JFK` : at each stop, try each unused ticket departing from the current airport, recurse, and undo if it doesn't lead to a full itinerary. This is trying all permutations of edges, so it's factorial and far too slow for anything but tiny inputs — but it's the natural starting point to state out loud.

```
public List<String> findItineraryBrute(List<List<String>> tickets) {
    List<String> route = new ArrayList<>();
    route.add("JFK");
    boolean[] used = new boolean[tickets.size()];
    if (backtrack(tickets, used, "JFK", route, tickets.size())) {
        return route;
    }
    return new ArrayList<>(); // no valid full itinerary (shouldn't happen if input
}

private boolean backtrack(List<List<String>> tickets, boolean[] used,
                          String from, List<String> route, int remaining) {
    if (remaining == 0) return true; // used every ticket

    // try destinations in lexical order so the first full itinerary found is the smallest
    List<Integer> candidates = new ArrayList<>();
    for (int i = 0; i < tickets.size(); i++) {
        if (!used[i] && tickets.get(i).get(0).equals(from)) candidates.add(i);
    }
    candidates.sort((a, b) -> tickets.get(a).get(1).compareTo(tickets.get(b).get(1)));

    for (int i : candidates) {
        used[i] = true;
        route.add(tickets.get(i).get(1));
        if (backtrack(tickets, used, tickets.get(i).get(1), route, remaining - 1)) return true;
        route.remove(route.size() - 1); // undo
        used[i] = false;
    }
    return false;
}
```

Optimized (Hierholzer's algorithm)

Build a map from airport to a min-heap of destinations (so the smallest airport always pops first). DFS greedily, consuming a ticket every time you leave a node. When a node runs out of outgoing tickets, it must be the end of the trail for that branch, so **prepend it to the result on the way back up** (post-order). Reversing the classic order gives the lexicographically smallest Eulerian path in one linear pass with no backtracking.

```

public List<String> findItinerary(List<List<String>> tickets) {
    Map<String, PriorityQueue<String>> graph = new HashMap<>();
    for (List<String> t : tickets) {
        graph.computeIfAbsent(t.get(0), k -> new PriorityQueue<>()).add(t.get(1));
    }

    LinkedList<String> route = new LinkedList<>();
    dfs("JFK", graph, route);
    return route; // already in correct order after prepending
}

private void dfs(String airport, Map<String, PriorityQueue<String>> graph, LinkedList<String>
    PriorityQueue<String> destinations = graph.get(airport);
    while (destinations != null && !destinations.isEmpty()) {
        String next = destinations.poll(); // always the smallest remaining destination
        dfs(next, graph, route);
    }
    route.addFirst(airport); // post-order: add once all outgoing tickets are used
}

```

Version	Time	Space
Brute force (backtracking over orderings)	$O(E!)$ worst case	$O(E)$ recursion
Hierholzer's (min-heap adjacency + DFS)	$O(E \log E)$	$O(E)$

If the interviewer pushes further: note that this only works because the graph is guaranteed to have an Eulerian path (at most one node with $\text{outdegree} - \text{indegree} = 1$, the rest balanced); mention you'd validate that precondition in a production setting rather than assume it.

What to say

"Ignoring the travel framing, this is 'find an Eulerian path' — a route that uses every directed edge exactly once, starting from a fixed node, with a lexicographically smallest tie-break. I'll use Hierholzer's algorithm: keep each node's outgoing edges in a min-heap so I always leave via the smallest airport, DFS greedily consuming edges, and prepend each node to the result once it has no edges left — that post-order reversal is what makes it Eulerian instead of just any spanning walk."

5. Recently-Used Cache

Evidence: [COMPANY-TAGGED — Booking.com (LeetCode 146, ~62.5% frequency)]

How the interviewer poses it. "Our booking service caches the hotels and pages a user has most recently viewed, so a repeat visit is instant instead of re-fetching from the database. The cache has a fixed capacity. Support `get(key)` — return the cached page/price, or -1 if absent — and `put(key, value)` — insert or update an entry. When the cache is full and a new key must be inserted, **evict the least recently used entry** first. Every access (get or put) should count as 'using' that key, refreshing its recency. Both operations must run in **$O(1)$** time."

De-story it

Strip the travel nouns:

```
nouns to ignore: "hotel", "page", "booking service", "user visit"
INPUT   : a capacity C, and a sequence of get(key) / put(key, value) calls
OUTPUT  : get returns the cached value or -1; put inserts/updates and,
          if over capacity, evicts the least-recently-used key
CONSTRAINT: get and put must both run in  $O(1)$  time and  $O(1)$  space per op
```

That is exactly **LeetCode 146: LRU Cache** — a fixed-capacity key-value store with constant-time lookup and constant-time recency tracking/eviction.

Worked example

```
capacity = 2

put(1, "hotelA")    cache (MRU->LRU): [1]
put(2, "hotelB")    cache: [2, 1]
get(1) -> "hotelA"  1 is used again -> moves to front: [1, 2]
put(3, "hotelC")    over capacity! evict LRU (key 2): [3, 1]
get(2) -> -1        (2 was evicted)
put(4, "hotelD")    over capacity! evict LRU (key 1): [4, 3]
get(1) -> -1        (1 was evicted)
get(3) -> "hotelC"
get(4) -> "hotelD"
```

Intuition (diagram)

HashMap gives $O(1)$ lookup; a doubly linked list gives $O(1)$ reordering.

```
key -> node lookup      recency order (MRU at head, LRU at tail)
map[1] --.             head <--> [1] <--> [3] <--> tail
map[3] ---+---> nodes  (get/put moves node to head;
map[4] --'             overflow evicts node before tail)
```

Brute force

HashMap for values, plus a separate timestamp/counter per key. To evict, **scan every key** to find the oldest timestamp — no ordering structure, so eviction costs $O(n)$.

```
class LRUCacheBrute {
    private final int capacity;
    private final Map<Integer, Integer> cache = new HashMap<>();
    private final Map<Integer, Integer> lastUsed = new HashMap<>();
    private int clock = 0;

    public LRUCacheBrute(int capacity) {
        this.capacity = capacity;
    }

    public int get(int key) {
        if (!cache.containsKey(key)) return -1;
        lastUsed.put(key, clock++); // touch = refresh recency
        return cache.get(key);
    }

    public void put(int key, int value) {
        if (!cache.containsKey(key) && cache.size() >= capacity) {
            int lruKey = -1, oldest = Integer.MAX_VALUE;
            for (Map.Entry<Integer, Integer> e : lastUsed.entrySet()) { //  $O(n)$  scan
                if (e.getValue() < oldest) { oldest = e.getValue(); lruKey = e.getKey(); }
            }
            cache.remove(lruKey);
            lastUsed.remove(lruKey);
        }
        cache.put(key, value);
        lastUsed.put(key, clock++);
    }
}
```

Optimized (HashMap + doubly linked list, $O(1)$)

Keep a `Node` per key in a doubly linked list ordered by recency, with sentinel `head / tail` nodes so splice/insert never touch null. The map gives $O(1)$ node lookup; moving a node to the front or unlinking the tail's neighbor is $O(1)$ pointer surgery — no scanning.

```

class LRUCache {
    private static class Node {
        int key, value;
        Node prev, next;
        Node(int key, int value) { this.key = key; this.value = value; }
    }

    private final int capacity;
    private final Map<Integer, Node> map = new HashMap<>();
    private final Node head = new Node(0, 0); // sentinel, head.next = MRU
    private final Node tail = new Node(0, 0); // sentinel, tail.prev = LRU

    public LRUCache(int capacity) {
        this.capacity = capacity;
        head.next = tail;
        tail.prev = head;
    }

    public int get(int key) {
        Node node = map.get(key);
        if (node == null) return -1;
        moveToFront(node);
        return node.value;
    }

    public void put(int key, int value) {
        Node node = map.get(key);
        if (node != null) {
            node.value = value;
            moveToFront(node);
            return;
        }
        if (map.size() == capacity) {
            Node lru = tail.prev; // node before sentinel tail
            unlink(lru);
            map.remove(lru.key);
        }
        Node fresh = new Node(key, value);
        map.put(key, fresh);
        insertAfterHead(fresh);
    }

    private void moveToFront(Node node) {
        unlink(node);
        insertAfterHead(node);
    }

    private void unlink(Node node) {
        node.prev.next = node.next;
        node.next.prev = node.prev;
    }

    private void insertAfterHead(Node node) {
        node.next = head.next;
        node.prev = head;
        head.next.prev = node;
        head.next = node;
    }
}

```

Operation	Brute force	Optimized
get	$O(1)$ lookup, $O(1)$ touch	$O(1)$
put (no eviction)	$O(1)$	$O(1)$
put (eviction)	$O(n)$ scan for LRU	$O(1)$
Space	$O(\text{capacity})$	$O(\text{capacity})$

Overlap note: this is the same HashMap + doubly-linked-list machinery you'd build for an LRU cache in a low-level design round (see Vol. 3). Here the only new skill is the **disguise recognition** — "recently-viewed hotels/pages with a size cap" is LRU eviction, nothing more.

What to say

"Past the booking framing, this is LeetCode 146: a fixed-capacity cache needing $O(1)$ get/put with least-recently-used eviction. A HashMap alone gives $O(1)$ lookup but no $O(1)$ ordering, so I pair it with a doubly linked list — sentinel head/tail so splicing never hits null — where the map stores key-to-node and the list encodes recency. Every get or put moves its node to the front; overflow evicts the node just before the tail. Both operations become $O(1)$."

6. Peak Price in Each Window

Evidence: [COMPANY-TAGGED — Booking.com (LeetCode 239, ~75% frequency; 100% in 6-month window)]

How the interviewer poses it. "We track the nightly **price** of a hotel room over a long stretch of days. A traveler is deciding on a **k-day stay** and wants to know, for every possible **k**-day window as it slides one day at a time across the whole calendar, what the **highest price** they'd be quoted during that stay would be. Given the array of daily prices and the stay length **k**, return the peak price for every window."

De-story it

Strip the travel nouns:

```
nouns to ignore: "hotel room", "nightly", "traveler", "stay"
INPUT    : an array of integers (the daily prices) and an integer k
           (the window size)
OUTPUT   : an array of length n-k+1, where result[i] is the max of
           the window prices[i .. i+k-1]
CONSTRAINT: the window slides by exactly one position at a time,
            1 <= k <= n
```

That is a **sliding window maximum**: for every contiguous window of fixed size **k**, report the max. Recomputing the max from scratch each time is wasteful — the classic optimization is a **monotonic decreasing deque of indices**, so the current window's max is always readable in $O(1)$.

Worked example

```
prices = [3, 1, 4, 6, 2, 5], k = 3

window [3,1,4] -> max 4
window [1,4,6] -> max 6
window [4,6,2] -> max 6
window [6,2,5] -> max 6

Answer: [4, 6, 6, 6]
```

Intuition (diagram)

Keep indices in the deque with strictly decreasing prices; front = window max.

```

i=0 push0: [3]
i=1 push1: [3,1]
i=2 pop1(1<4) pop0(3<4) push2: [4]      max=4
i=3 pop2(4<6) push3: [6]                max=6
i=4 push4: [6,2]                        max=6
i=5 pop4(2<5) push5: [6,5] drop0<=win: [6,5] max=6

```

Brute force

For each window, scan all `k` elements to find the max.

```

public int[] peakPricesBrute(int[] prices, int k) {
    int n = prices.length;
    int[] result = new int[n - k + 1];
    for (int i = 0; i <= n - k; i++) {          // one scan per window
        int max = prices[i];
        for (int j = i + 1; j < i + k; j++) {
            max = Math.max(max, prices[j]);
        }
        result[i] = max;
    }
    return result;
}

```

Optimized (monotonic decreasing deque of indices)

Keep a deque of indices whose prices are in strictly decreasing order. Before adding a new index, pop smaller prices off the back (they can never be the max again). Before reading the max, pop indices that fell out of the window off the front. The front is always the current window's max.

```

public int[] peakPrices(int[] prices, int k) {
    int n = prices.length;
    int[] result = new int[n - k + 1];
    Deque<Integer> deque = new ArrayDeque<>(); // holds indices, decreasing prices

    for (int i = 0; i < n; i++) {
        // drop indices whose price can't beat prices[i]; they'll never be reported
        while (!deque.isEmpty() && prices[deque.peekLast()] <= prices[i]) {
            deque.pollLast();
        }
        deque.offerLast(i);

        // front index has slid out of the current window
        if (deque.peekFirst() <= i - k) {
            deque.pollFirst();
        }

        // first full window is complete once we've seen k elements
        if (i >= k - 1) {
            result[i - k + 1] = prices[deque.peekFirst()];
        }
    }
    return result;
}

```

Version	Time	Space
Brute force (rescan every window)	$O(n \cdot k)$	$O(1)$ extra
Monotonic deque	$O(n)$	$O(k)$

What to say

"Ignoring the pricing framing, this is sliding window maximum: report the max of every fixed-size window as it slides by one. Instead of rescanning each window, I keep a monotonic decreasing deque of indices — its front is always the current window's max. On each step I pop smaller values off the back since they can never win again, push the new index, then pop the front if it's aged out of the window. That gives $O(n)$ instead of $O(n \cdot k)$."

7. Escape the Warehouse Grid

Evidence: [COMPANY-TAGGED — UiPath (LeetCode 1036, "Escape a Large Maze")]

How the interviewer poses it. "An automation bot sits in a massive warehouse laid out as a grid — think $10^6 \times 10^6$ storage cells. The bot needs to travel from its `source` cell to a `target` cell, moving one cell at a time in the four cardinal directions. A handful of cells are marked as **blocked** (obstacles, roped-off zones — at most a couple hundred of them). Given the blocked cells, `source`, and `target`, can the bot actually reach the target?"

De-story it

Strip the warehouse nouns:

```
nouns to ignore: "bot", "warehouse", "storage cells", "roped-off zones"
INPUT   : a huge grid (up to  $10^6 \times 10^6$ ), a small list of blocked
          cells (at most ~200), a source cell, a target cell
OUTPUT  : true if source can reach target via 4-directional moves
          through unblocked cells, false otherwise
CONSTRAINT: grid is far too large to allocate or fully traverse;
          the number of blocked cells B is tiny ( $B \leq 200$ )
```

That is a **reachability-with-bounded-obstacles** problem: the grid itself is irrelevant — what matters is that so few blockers can only wall off a small pocket of space. Once you realize the *enclosable area* is capped by how many blockers exist, this becomes a **bounded BFS** problem, not a "search the whole grid" problem.

Worked example

```
grid: conceptually 1,000,000 x 1,000,000
blocked = [[0,1],[1,0]]      (2 blockers)
source  = [0,0]
target  = [0,2]
```

Around source (0,0):

```
(0,1) blocked, (1,0) blocked -> source is boxed into a corner
the only free neighbors from (0,0) are... none (both blocked)
max area 2 blockers can enclose =  $2 \times (2-1) / 2 = 1$  cell
BFS from source explores 1 cell (itself) and stops -> "trapped"
```

Answer: false (source can never escape its 1-cell pocket)

```
If instead blocked = [[0,1]] only (1 blocker), source=[0,0], target=[2,2]:
max enclosable area =  $1 \times (1-1) / 2 = 0$  -> a single blocker can't
wall anything off, so BFS from source immediately exceeds the
bound (or reaches target) -> true
```

Intuition (diagram)

B blockers can wall off at most $B*(B-1)/2$ cells into a pocket.
BFS a bounded number of steps; if you escape the pocket, you're free.

. . # . .	# = blocked	S = source	T = target
. S # . .	BFS from S expands outward; if the frontier		
. . # . .	count ever exceeds $B*(B-1)/2$, S is NOT boxed in		
. . # T .	(no ring of blockers that small can seal it off)		
. . # . .			

Brute force

Run a normal BFS/DFS over the **entire** grid from `source`, marking visited cells, until `target` is found or the queue empties.

```
public boolean isEscapePossibleBrute(int[][] blocked, int[] source, int[] target) {
    int n = 1_000_000;
    boolean[][] visited = new boolean[n][n];          // impossible: 10^12 cells
    Set<String> block = new HashSet<>();
    for (int[] b : blocked) block.add(b[0] + "," + b[1]);

    Deque<int[]> queue = new ArrayDeque<>();
    queue.add(source);
    visited[source[0]][source[1]] = true;
    int[][] dirs = {{0, 1}, {0, -1}, {1, 0}, {-1, 0}};

    while (!queue.isEmpty()) {
        int[] cur = queue.poll();
        if (cur[0] == target[0] && cur[1] == target[1]) return true;
        for (int[] d : dirs) {
            int r = cur[0] + d[0], c = cur[1] + d[1];
            if (r < 0 || r >= n || c < 0 || c >= n) continue;
            if (visited[r][c] || block.contains(r + "," + c)) continue;
            visited[r][c] = true;
            queue.add(new int[]{r, c});
        }
    }
    return false;
}
```

This allocates a $10^6 \times 10^6$ visited array — **10^{12} cells**, far beyond any reasonable memory or time budget. It never finishes; it's not a "slow" solution, it's an infeasible one.

Optimized (bounded BFS from both ends)

Only up to ~200 blockers exist, so they can enclose at most $B*(B-1)/2$ cells (the largest area a ring of B cells can seal off on a grid). Run a BFS from `source` capped at that many visited cells: if the BFS runs out of frontier before hitting the cap, `source` is trapped in a small pocket that (by the cap) cannot contain `target` reachably beyond it — but to be correct we also must confirm `target` isn't trapped in *its own* tiny pocket. So do the same bounded BFS from `target`. Both bots "escape" their local neighborhoods (exceed the cap) $\Rightarrow$ they're both in the same unbounded open region $\Rightarrow$ reachable.

```

public boolean isEscapePossible(int[][] blocked, int[] source, int[] target) {
    int n = 1_000_000;
    Set<Long> block = new HashSet<>();
    for (int[] b : blocked) block.add(encode(b[0], b[1], n));

    int limit = blocked.length * (blocked.length - 1) / 2;    // max enclosable area

    // Each side must either reach the other point, or escape its bounded pocket.
    return boundedBfs(source, target, block, n, limit)
        && boundedBfs(target, source, block, n, limit);
}

private boolean boundedBfs(int[] start, int[] goal, Set<Long> block, int n, int limit) {
    if (limit == 0) return true;    // no blockers can enclose anything

    Set<Long> visited = new HashSet<>();
    Deque<int[]> queue = new ArrayDeque<>();
    queue.add(start);
    visited.add(encode(start[0], start[1], n));
    int[][] dirs = {{0, 1}, {0, -1}, {1, 0}, {-1, 0}};

    while (!queue.isEmpty()) {
        int[] cur = queue.poll();
        if (cur[0] == goal[0] && cur[1] == goal[1]) return true;    // reached the other end
        if (visited.size() > limit) return true;    // escaped the bounded pock

        for (int[] d : dirs) {
            int r = cur[0] + d[0], c = cur[1] + d[1];
            if (r < 0 || r >= n || c < 0 || c >= n) continue;
            long key = encode(r, c, n);
            if (visited.contains(key) || block.contains(key)) continue;
            visited.add(key);
            queue.add(new int[]{r, c});
        }
    }
    return false;    // BFS exhausted without reaching goal or escaping – truly boxed in
}

private long encode(int r, int c, int n) {
    return (long) r * n + c;
}

```

Version	Time	Space
Brute force (full-grid BFS)	$O(n^2)$ — infeasible at $n=10^6$	$O(n^2)$ — infeasible
Bounded BFS from both ends	$O(B^2)$	$O(B^2)$

If the interviewer pushes further: note the $B*(B-1)/2$ bound comes from the fact that a ring of B cells forming a diamond/box around a corner encloses at most that many cells; any BFS that visits more cells than that *must* have broken free of any possible enclosure, so capping the visited-set size is both a correctness argument and a performance guarantee.

What to say

"Ignoring the warehouse framing, this is reachability on a huge grid with only a handful of obstacles. A full BFS over $10^6 \times 10^6$ cells is impossible, but since at most B blockers exist, they can only wall off about $B(B-1)/2$ cells. So I'll BFS from source capped at that area — if it escapes the cap, it's in the open region. I'll do the same from target, and return true only if both escape their local pockets (or one directly reaches the other). That gives $O(B^2)$ time and space instead of scanning the whole grid."

8. Biggest Connected Zone

Evidence: [COMPANY-TAGGED — UiPath (LeetCode 827, "Making A Large Island")]

How the interviewer poses it. "We have a warehouse floor plan represented as a grid of **automation zones**. Each cell is either **1** (an active robot cell) or **0** (empty floor). You have budget to **activate at most one empty cell** — flip a single **0** to a **1** — to extend your automation coverage. Active cells that touch **up, down, left, or right** are considered part of the same connected zone. After your one activation, what is the **size of the largest connected zone** you can achieve?"

De-story it

Strip the warehouse nouns:

```
nouns to ignore: "warehouse", "robot cell", "automation zone"
INPUT    : an m x n binary grid (0s and 1s)
OUTPUT   : the max size of a 4-directionally-connected group of 1s,
           after flipping at most one 0 to a 1
CONSTRAINT: at most one flip; grid may already be all 1s
```

That is a **connected components / union-find (or DFS-labeling)** problem: identify existing islands and their sizes, then figure out which single flip would merge the most neighboring islands together.

Worked example

```
grid:
  1 1 0
  1 0 0
  0 0 1

label islands:
  island A = {(0,0),(0,1),(1,0)} size 3
  island B = {(2,2)} size 1

candidate flips (each 0 cell):
  (0,2): neighbors -> A(3)           => 3 + 1 = 4
  (1,1): neighbors -> A(3)           => 3 + 1 = 4
  (1,2): neighbors -> B(1)           => 1 + 1 = 2
  (2,0): neighbors -> (none)         => 0 + 1 = 1
  (2,1): neighbors -> B(1)           => 1 + 1 = 2
Answer: 4 (flip (0,2) or (1,1))
```

Intuition (diagram)

```
grid ids:          flip a 0, look at its up/down/left/right ids,
A A .              sum the DISTINCT island sizes touched, +1 (self)
A . .
. . B
(1,1) touches only A(3) -> 3+1=4 <- best
```

Brute force

For every empty cell, flip it, re-scan the whole grid with DFS to measure the largest island, then unflip. Simple but repeats full-grid work for every candidate.

```
public int largestZoneBrute(int[][] grid) {
    int m = grid.length, n = grid[0].length;
    int best = 0;
    boolean anyZero = false;

    for (int r = 0; r < m; r++) {
        for (int c = 0; c < n; c++) {
            if (grid[r][c] == 1) continue;
            anyZero = true;
            grid[r][c] = 1; // flip
            boolean[][] seen = new boolean[m][n];
            for (int i = 0; i < m; i++) {
                for (int j = 0; j < n; j++) {
                    if (grid[i][j] == 1 && !seen[i][j]) {
                        best = Math.max(best, dfsSize(grid, seen, i, j));
                    }
                }
            }
            grid[r][c] = 0; // unflip
        }
    }
    if (!anyZero) return m * n; // already all 1s
    return best;
}

private int dfsSize(int[][] grid, boolean[][] seen, int r, int c) {
    int m = grid.length, n = grid[0].length;
    if (r < 0 || r >= m || c < 0 || c >= n) return 0;
    if (seen[r][c] || grid[r][c] == 0) return 0;
    seen[r][c] = true;
    return 1 + dfsSize(grid, seen, r + 1, c) + dfsSize(grid, seen, r - 1, c)
        + dfsSize(grid, seen, r, c + 1) + dfsSize(grid, seen, r, c - 1);
}
```

Optimized (label islands once, then evaluate each flip)

Two passes over the grid instead of one full re-scan per candidate: 1. DFS every existing `1`, giving each island an id and recording `id -> size` in a map. 2. For every `0`, look at its up to 4 neighbors, collect the **distinct** island ids touched (a `Set` dedupes an island counted twice, e.g. two neighbors from the same island), sum their sizes, add 1 for the flipped cell itself, and track the max.

```

public int largestZone(int[][] grid) {
    int m = grid.length, n = grid[0].length;
    int[][] id = new int[m][n];           // 0 = unvisited, else island id
    Map<Integer, Integer> idToSize = new HashMap<>();
    int nextId = 2;                       // start above 0/1 to avoid clashing

    for (int r = 0; r < m; r++) {
        for (int c = 0; c < n; c++) {
            if (grid[r][c] == 1 && id[r][c] == 0) {
                int size = label(grid, id, r, c, nextId);
                idToSize.put(nextId, size);
                nextId++;
            }
        }
    }

    int best = idToSize.values().stream().mapToInt(Integer::intValue).max().orElse(0);
    boolean anyZero = false;

    for (int r = 0; r < m; r++) {
        for (int c = 0; c < n; c++) {
            if (grid[r][c] != 0) continue;
            anyZero = true;
            Set<Integer> neighborIds = new HashSet<>();
            addIfIsland(id, r - 1, c, m, n, neighborIds);
            addIfIsland(id, r + 1, c, m, n, neighborIds);
            addIfIsland(id, r, c - 1, m, n, neighborIds);
            addIfIsland(id, r, c + 1, m, n, neighborIds);

            int total = 1;                  // the flipped cell itself
            for (int nid : neighborIds) total += idToSize.get(nid);
            best = Math.max(best, total);
        }
    }

    return anyZero ? best : m * n;         // all 1s already -> whole grid
}

private int label(int[][] grid, int[][] id, int r, int c, int markId) {
    int m = grid.length, n = grid[0].length;
    if (r < 0 || r >= m || c < 0 || c >= n) return 0;
    if (grid[r][c] == 0 || id[r][c] != 0) return 0;
    id[r][c] = markId;
    return 1 + label(grid, id, r + 1, c, markId) + label(grid, id, r - 1, c, markId)
        + label(grid, id, r, c + 1, markId) + label(grid, id, r, c - 1, markId);
}

private void addIfIsland(int[][] id, int r, int c, int m, int n, Set<Integer> out) {
    if (r < 0 || r >= m || c < 0 || c >= n) return;
    if (id[r][c] != 0) out.add(id[r][c]);
}

```

Version	Time	Space
Brute force (flip + full DFS each time)	$O((m \cdot n)^2)$	$O(m \cdot n)$
Label once + evaluate each flip	$O(m \cdot n)$	$O(m \cdot n)$

If the interviewer asks for an alternative: union-find works just as well for the labeling pass — union adjacent 1s, then read `find(x)` and component size for each candidate flip. Same overall complexity,

useful if the grid is streamed or updated incrementally.

What to say

"Past the warehouse framing, this is connected components on a binary grid: find existing islands and their sizes, then for each empty cell sum the sizes of the *distinct* islands it touches, plus one for itself. I'll label islands with DFS in one pass — $O(m \cdot n)$ — instead of re-scanning the whole grid for every candidate flip, which is what the brute force does and costs $O((m \cdot n)^2)$. I need a set when combining neighbor sizes so a diagonal-adjacent-looking island touched twice isn't double-counted, and I need to special-case an all-1s grid where there's no 0 left to flip."

9. Consistent Rules Check

Evidence: [COMPANY-TAGGED — UiPath (LeetCode 990, "Satisfiability of Equality Equations")]

How the interviewer poses it. "Our RPA rule engine lets an automation designer attach **variable rules** to a workflow, things like `a==b` or `a!=c`, where each rule says two variables must either be **equal** or **not equal**. Before we compile the workflow, we need to know whether the whole rule set can be satisfied **simultaneously** — i.e., is there any assignment of values to the variables that makes every rule true at once, or do the rules contradict each other?"

De-story it

Strip the RPA nouns:

```
nouns to ignore: "workflow", "automation designer", "rule engine"
INPUT    : a list of strings, each of the form "xx==yy" or "xx!=yy",
           where xx and yy are single lowercase-letter variable names
OUTPUT   : true if some assignment of values satisfies every rule,
           false if any rule contradicts the others
CONSTRAINT: "==" rules must chain (equality is transitive);
            a single "!=" rule that connects two already-equal
            variables makes the whole set unsatisfiable
```

That is a **connectivity / grouping** problem: `==` glues variables into groups where every member must share one value; `!=` is a constraint that two variables must land in **different** groups. That's exactly what **union-find (disjoint set union)** models — or, equivalently, graph connectivity where you check group membership.

Worked example

```
equations = ["a==b", "b==c", "c!=a"]

pass 1 (union all "==" pairs):
  a==b -> union(a, b)      groups: {a,b}, {c}
  b==c -> union(b, c)      groups: {a,b,c}

pass 2 (check every "!=" pair):
  c!=a -> find(c) == find(a)? YES (same group) -> contradiction!

Answer: false
```

```
equations = ["a==b", "b!=c", "c==a"]
```

```
pass 1: a==b -> {a,b}
        c==a -> {a,b,c}      (a and c forced into the same group)
```

```
pass 2: b!=c -> find(b) == find(c)? YES (same group) -> contradiction!
```

```
Answer: false
```

Intuition (diagram)

`==` merges circles into one group; `!=` demands two circles differ.

```
a==b, b==c      group: (a)-(b)-(c)   one root
c!=a  ----> same root as a --> CONTRADICTION, return false
```

Approach 1 (graph connectivity)

Build a graph: an edge for every `==` pair. Run BFS/DFS to label each connected component. Then for every `!=` pair, check the two variables landed in **different** components; if any pair shares a component, return false.

```

public boolean equationsPossibleGraph(String[] equations) {
    Map<Character, List<Character>> adj = new HashMap<>();
    for (char c = 'a'; c <= 'z'; c++) adj.put(c, new ArrayList<>());

    List<char[]> notEqual = new ArrayList<>();
    for (String eq : equations) {
        char x = eq.charAt(0), y = eq.charAt(3);
        if (eq.charAt(1) == '=') {                // "xx==yy"
            adj.get(x).add(y);
            adj.get(y).add(x);
        } else {                                // "xx!=yy"
            notEqual.add(new char[]{x, y});
        }
    }

    int[] comp = new int[26];
    Arrays.fill(comp, -1);
    int id = 0;
    for (char c = 'a'; c <= 'z'; c++) {        // BFS to label components
        if (comp[c - 'a'] != -1) continue;
        Deque<Character> queue = new ArrayDeque<>();
        queue.add(c);
        comp[c - 'a'] = id;
        while (!queue.isEmpty()) {
            char cur = queue.poll();
            for (char nxt : adj.get(cur)) {
                if (comp[nxt - 'a'] == -1) {
                    comp[nxt - 'a'] = id;
                    queue.add(nxt);
                }
            }
        }
        id++;
    }

    for (char[] pair : notEqual) {
        if (comp[pair[0] - 'a'] == comp[pair[1] - 'a']) return false;    // forced equal but not
    }
    return true;
}

```

Approach 2 (union-find, optimized)

Same idea, but skip building an explicit graph. Union every `==` pair directly; then for every `!=` pair, `find` both variables and reject if they share a root. Path compression + union by rank keep every operation nearly $O(1)$.

```

private int[] parent = new int[26];
private int[] rank_ = new int[26];

public boolean equationsPossible(String[] equations) {
    for (int i = 0; i < 26; i++) parent[i] = i;    // 26 lowercase letters

    for (String eq : equations) {                // pass 1: union all "=="
        if (eq.charAt(1) == '=') {
            union(eq.charAt(0) - 'a', eq.charAt(3) - 'a');
        }
    }

    for (String eq : equations) {                // pass 2: check every "!="
        if (eq.charAt(1) == '!') {
            int x = eq.charAt(0) - 'a', y = eq.charAt(3) - 'a';
            if (find(x) == find(y)) return false; // contradiction: forced equal, marked unequal
        }
    }
    return true;
}

private int find(int x) {
    if (parent[x] != x) parent[x] = find(parent[x]); // path compression
    return parent[x];
}

private void union(int x, int y) {
    int rx = find(x), ry = find(y);
    if (rx == ry) return;
    if (rank_[rx] < rank_[ry]) {                  // union by rank
        parent[rx] = ry;
    } else if (rank_[rx] > rank_[ry]) {
        parent[ry] = rx;
    } else {
        parent[ry] = rx;
        rank_[rx]++;
    }
}

```

Version	Time	Space
Approach 1 (graph BFS/DFS)	$O(n + 26)$	$O(26)$
Approach 2 (union-find, path compression + rank)	$O(n \cdot \alpha(26)) \approx O(n)$	$O(26)$

Note on scale: the canonical version restricts variables to single lowercase letters (26 possible), so both approaches are effectively linear in the number of rules. If the interviewer generalizes to arbitrary variable names, swap the fixed `int[26]` arrays for a `Map<String, String>` parent map — the union-find logic is unchanged.

What to say

"Underneath the RPA framing, this is: build groups of variables forced equal by `==`, then check no `!=` rule connects two variables in the same group. I'll union every `==` pair with union-find, then walk the `!=` rules and reject if `find` returns the same root for both sides.

Path compression and union by rank keep it near-linear, and since variables are single lowercase letters I can back the structure with a fixed-size array instead of a map."

10. Prune the Bot Task Tree

Evidence: [COMPANY-TAGGED — UiPath (LeetCode 2603, "Collect Coins in a Tree" — graph / leaf-trimming / topological pruning)]

How the interviewer poses it. "Our RPA bots execute a **task tree**: n tasks are wired together by $n - 1$ dependency links, so the whole thing forms a tree. Some tasks carry a **reward credit** (a coin). A bot doesn't have to physically step onto a rewarded task to collect its credit — it can harvest any reward that is **within 2 hops** of wherever it is currently standing. The bot starts and must end at task 0 (home base). Every dependency link it walks, it eventually has to walk back (round trip). What is the **minimum number of links the bot must traverse** to collect every reward credit in the tree and return home?"

De-story it

Strip the automation nouns:

```
nouns to ignore: "bot", "task", "dependency link", "reward credit", "home base"
INPUT    : an undirected tree of  $n$  nodes ( $n-1$  edges), plus a  $0/1$  array
           marking which nodes hold a "coin"
OUTPUT   : the minimum number of edges that must be traversed, starting
           and ending at node  $0$ , to end up within distance 2 of every
           coin node at some point along the walk
CONSTRAINT:  $n$  up to  $3 \cdot 10^4$ , tree (no cycles), must return to the start
```

This is a **graph-pruning** problem, not a pathfinding one: because it's a tree, any node you never need to *stand on* (no coin nearby depends on it) is deadweight — chop it off. Once you strip away every branch that leads to nothing useful, what's left is the minimal "spine" you must actually walk, and a round trip means walking each surviving edge twice.

Worked example

6 tasks in a line, coins (*) at the two ends:

0*---1---2---3---4---5*

Phase 1 – trim leaves with NO coin: both leaves (0 and 5) HAVE coins, so nothing trims yet. Remaining edges: 5

Phase 2 – trim leaves for 2 more rounds (coins within distance 2 don't need to be stood on):

round 1: trim leaves {0, 5}	-> left with	1---2---3---4	(3 edges)
round 2: trim leaves {1, 4}	-> left with	2---3	(1 edge)

1 edge survives -> answer = $2 \cdot 1 = 2$

The bot never needs to walk out to node 0 or node 5 at all — standing at node 2 or node 3 is already within distance 2 of both coins.

Intuition (diagram)

```
0*--1--2--3--4--5*    * = coin, must be within dist 2 to collect
trim {0,5}: 1--2--3--4
trim {1,4}:    2--3
1 edge left -> 2*1 = 2
```

A brute-force answer would try every possible walk (or every subset of nodes to visit) and check coin coverage by simulation — that's exponential and pointless on a tree. Trees give you a much sharper tool: **iterative leaf pruning**, the same "peel it from the outside in" idea behind topological sort (Kahn's algorithm), except here we peel degree-1 nodes instead of zero-indegree nodes.

Approach (leaf-trimming / topological pruning)

1. Build an adjacency list and a `degree[]` array from the edges.
2. **Trim useless leaves:** queue every leaf (`degree == 1`) that has no coin, then repeatedly pop a leaf, delete its edge (decrement the neighbor's degree), and if the neighbor *becomes* a coin-less leaf, queue it too. This removes every branch that never reaches a coin — like Kahn's algorithm, but the "ready to remove" condition is "leaf with no coin" instead of "indegree zero".
3. **Trim two more outer layers**, leaves only, coin or not: a coin sitting on a leaf that survived phase 1 can still be collected from one or two hops away, so you never need to walk onto it or its immediate neighbor.
4. Whatever edges remain form the minimal spine the bot must actually walk. Since it's a round trip, the answer is `2 * remainingEdges`.

```

public int minEdges(int[] coins, int[][] edges) {
    int n = coins.length;
    if (n <= 1) return 0; // nothing to walk

    List<List<Integer>> adj = new ArrayList<>();
    for (int i = 0; i < n; i++) adj.add(new ArrayList<>());
    int[] degree = new int[n];
    for (int[] e : edges) {
        adj.get(e[0]).add(e[1]);
        adj.get(e[1]).add(e[0]);
        degree[e[0]]++;
        degree[e[1]]++;
    }
    int remainingEdges = edges.length;

    // Phase 1: repeatedly trim leaves that carry no coin (Kahn-style peel).
    Deque<Integer> queue = new ArrayDeque<>();
    for (int i = 0; i < n; i++) {
        if (degree[i] == 1 && coins[i] == 0) queue.add(i);
    }
    while (!queue.isEmpty()) {
        int leaf = queue.poll();
        degree[leaf] = 0;
        remainingEdges--;
        for (int nb : adj.get(leaf)) {
            if (degree[nb] == 0) continue;
            degree[nb]--;
            if (degree[nb] == 1 && coins[nb] == 0) queue.add(nb);
        }
    }

    // Phase 2: trim two more outer layers of leaves, coin or not --
    // those coins are collectible from distance <= 2 without stepping on them.
    for (int round = 0; round < 2; round++) {
        List<Integer> leaves = new ArrayList<>();
        for (int i = 0; i < n; i++) {
            if (degree[i] == 1) leaves.add(i);
        }
        for (int leaf : leaves) {
            if (degree[leaf] == 0) continue;
            degree[leaf] = 0;
            remainingEdges--;
            for (int nb : adj.get(leaf)) {
                if (degree[nb] > 0) degree[nb]--;
            }
        }
    }

    return Math.max(remainingEdges, 0) * 2;
}

```

Approach	Time	Space
Brute force (simulate every walk / node subset)	exponential	—
Leaf-trimming / topological pruning	O(n)	O(n) adjacency + degree

What to say

"This is a tree, so instead of searching paths I prune from the leaves inward, like Kahn's algorithm but trimming degree-1 nodes with no coin instead of zero-indegree nodes. First I strip every coin-less dead-end branch, then I strip two more outer layers of leaves because a coin within distance 2 doesn't require standing on it. Whatever edges survive form the minimal spine, and since it's a round trip, the answer is twice that edge count — all $O(n)$."

11. Critical Network Links

Evidence: [CANONICAL — the standard networking analog; LeetCode 1192 "Critical Connections in a Network", LeetCode's official hint is "Use Tarjan's algorithm"]

How the interviewer poses it. "You run a data center with n servers, numbered 0 to $n-1$, wired together by $n-1$ -or-more direct **network links** (undirected — traffic flows both ways). A link is **critical** if removing it would leave at least one server unable to reach at least one other server — i.e., it would **partition the network**. Given the list of links, return **all critical links**."

De-story it

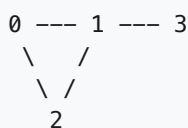
Strip the data-center nouns:

```
nouns to ignore: "servers", "data center", "network links", "traffic"
INPUT    : n (number of nodes), and a list of undirected edges connecting them
OUTPUT   : every edge whose removal increases the number of connected
            components (splits the graph into more pieces)
CONSTRAINT: graph is connected (or you only care about edges within
            each connected component); no self-loops, no duplicate edges
```

That is exactly the definition of a graph **bridge**: an edge that is not part of any cycle. Any edge sitting on a cycle has an alternate path around it, so removing it can't disconnect anything — it's the acyclic, "single point of failure" edges you're after.

Worked example

```
n = 4, links = [[0,1],[1,2],[2,0],[1,3]]
```



0-1, 1-2, 2-0 form a triangle (a cycle) → none of them is critical, each has a detour around it.

1-3 is the only way to reach server 3 → removing it isolates 3.

Answer: `[[1,3]]` (the single bridge)

Intuition (diagram)

```
0===1---3    ===: on a cycle, safe    ---: a bridge, critical
 \   /
  \ /
   2
```

One DFS pass tags each node with a **discovery time** (`disc`, when it was first visited) and a **low-link value** (`low`, the earliest discovery time reachable from that node's subtree using at most one back-edge — i.e., without walking back up through the edge to its parent). If a child `v` can't reach anything discovered at or before its parent `u` (`low[v] > disc[u]`), then the edge `(u,v)` is the only bridge holding `v`'s subtree onto the rest of the graph.

Brute force

For every edge, remove it and check with a fresh DFS/BFS whether the graph is still fully (pairwise) connected; if not, it was critical.

```
public List<List<Integer>> criticalBrute(int n, List<List<Integer>> edges) {
    List<List<Integer>> bridges = new ArrayList<>();
    for (int i = 0; i < edges.size(); i++) {
        List<Integer> removed = edges.get(i);
        List<List<Integer>> remaining = new ArrayList<>(edges);
        remaining.remove(i);
        if (!isConnected(n, remaining)) {
            bridges.add(removed);
        }
    }
    return bridges;
}

private boolean isConnected(int n, List<List<Integer>> edges) {
    Map<Integer, List<Integer>> adj = new HashMap<>();
    for (int i = 0; i < n; i++) adj.put(i, new ArrayList<>());
    for (List<Integer> e : edges) {
        adj.get(e.get(0)).add(e.get(1));
        adj.get(e.get(1)).add(e.get(0));
    }
    boolean[] seen = new boolean[n];
    Deque<Integer> stack = new ArrayDeque<>();
    stack.push(0);
    seen[0] = true;
    int count = 1;
    while (!stack.isEmpty()) {
        int cur = stack.pop();
        for (int next : adj.get(cur)) {
            if (!seen[next]) {
                seen[next] = true;
                count++;
                stack.push(next);
            }
        }
    }
    return count == n; // every node still reachable from 0
}
```

Optimized (Tarjan's bridge-finding, single DFS)

Track `disc[]` (discovery time) and `low[]` (lowest discovery time reachable) with one ticking timer, remembering the parent so we don't treat the edge we just walked down as a back-edge.

```

private List<List<Integer>> adj;
private int[] disc, low;
private int timer;
private List<List<Integer>> bridges;

public List<List<Integer>> criticalConnections(int n, List<List<Integer>> edges) {
    adj = new ArrayList<>();
    for (int i = 0; i < n; i++) adj.add(new ArrayList<>());
    for (List<Integer> e : edges) {
        adj.get(e.get(0)).add(e.get(1));
        adj.get(e.get(1)).add(e.get(0));
    }
    disc = new int[n];
    low = new int[n];
    Arrays.fill(disc, -1); // -1 = unvisited
    bridges = new ArrayList<>();
    timer = 0;
    dfs(0, -1);
    return bridges;
}

private void dfs(int u, int parent) {
    disc[u] = low[u] = timer++;
    for (int v : adj.get(u)) {
        if (v == parent) continue; // skip the edge we just arrived on
        if (disc[v] == -1) { // v not yet visited -> tree edge
            dfs(v, u);
            low[u] = Math.min(low[u], low[v]);
            if (low[v] > disc[u]) { // v's subtree can't reach back past u
                bridges.add(Arrays.asList(u, v));
            }
        } else { // back edge to an ancestor
            low[u] = Math.min(low[u], disc[v]);
        }
    }
}
}

```

Version	Time	Space
Brute force (remove each edge, re-check connectivity)	$O(E \cdot (V + E))$	$O(V + E)$
Tarjan's (single DFS with disc/low)	$O(V + E)$	$O(V + E)$ recursion + adjacency

What to say

"Ignoring the data-center framing, this is: find every bridge in an undirected graph — an edge that isn't on any cycle. Brute force removes each edge and re-checks connectivity, which is $O(E \cdot (V + E))$. I'll do it in one DFS instead: track discovery time and low-link value per node, and flag edge (u,v) as a bridge when `low[v] > disc[u]`, meaning v's subtree has no back-edge past u. That's Tarjan's bridge algorithm, $O(V + E)$."

Appendix A. IP Routing via Trie (conceptual — not a reported question)

Evidence: [CONCEPTUAL — NOT a reported interview question.] Our verified research found NO evidence that Booking.com, Airbnb, Microsoft, Agoda, or UiPath has asked a trie-based IP-routing / CIDR / longest-prefix-match question. It is included here only for learning value because it is a genuinely elegant trie application. Do NOT claim in an interview that any specific company asks this.

The concept. Routers forward packets using **longest-prefix match (LPM)**. Every IP address is just a 32-bit number. A routing table doesn't store individual addresses — it stores **prefixes** (written as CIDR blocks like `10.1.0.0/16`, meaning "the first 16 bits are fixed, the rest can be anything"). When a packet arrives for some destination IP, the router looks at all the prefixes it knows and picks the **most specific one that matches** — i.e., the prefix with the largest number of fixed leading bits that still agrees with the address. More specific routes win because they represent more precise knowledge about how to reach that destination.

De-story it

```
nouns to ignore: "router", "packet", "network", "CIDR block"
INPUT   : a set of binary prefixes (each is a bit-string of length <= 32,
          e.g. "00001010" for 10.0.0.0/8), plus a 32-bit address to look up
OUTPUT  : the prefix in the set that (a) matches the address's leading bits
          and (b) is the LONGEST such prefix
CONSTRAINT: multiple prefixes may match the same address; longer wins
```

That is a **prefix-matching / trie** problem: insert every prefix into a tree keyed bit-by-bit, then walk the tree along the address's bits, remembering the deepest "this is a valid prefix end" marker you pass through.

Worked example

```
Prefixes in the table:
10.0.0.0/8    -> bits: 00001010                (8 bits fixed)
10.1.0.0/16   -> bits: 0000101000000001        (16 bits fixed)

Lookup address: 10.1.2.3
binary:        00001010.00000001.00000010.00000011

Both prefixes match the first bits of the address:
00001010      matches (that's 10.0.0.0/8)
0000101000000001 matches (that's 10.1.0.0/16)

10.1.0.0/16 is longer (16 fixed bits vs 8), so it wins.
Answer: 10.1.0.0/16
```

Intuition (diagram)

Each prefix is a path from the root; a lookup walks the same bits and remembers the last "prefix ends here" flag it passed.

```
      root
    0/   \1
    *      (10.x = 00001010...)
  0/   \1
... node marks end of 10.0.0.0/8
   |0
   node marks end of 10.1.0.0/16
lookup walks 0,0,0,0,1,0,1,0,0,0,0,0,0,0,0,1...
remembers deepest "end of prefix" seen -> /16 wins
```

Approach (binary trie / longest-prefix match)

Build a binary trie where each node has at most two children (0 and 1). Insert a prefix by walking its bits and creating nodes as needed, then flag the final node as "end of prefix" (also storing the prefix's own identity for reporting). To look up an address, walk its 32 bits from the most significant down; every time you pass through a node flagged "end of prefix," remember it — that's your best answer so far. Stop when you run out of trie nodes or bits; the last remembered flag is the longest match.

```

public class LpmTrie {

    private static class Node {
        Node[] children = new Node[2]; // 0 and 1
        boolean isPrefixEnd = false;
        String prefixLabel;           // e.g. "10.1.0.0/16", for reporting
    }

    private final Node root = new Node();

    /** Insert a CIDR-style prefix given as its fixed high bits and length (0..32). */
    public void insert(int addressBits, int prefixLen, String label) {
        Node cur = root;
        for (int i = 0; i < prefixLen; i++) {
            int bit = (addressBits >>> (31 - i)) & 1;
            if (cur.children[bit] == null) {
                cur.children[bit] = new Node();
            }
            cur = cur.children[bit];
        }
        cur.isPrefixEnd = true;
        cur.prefixLabel = label;
    }

    /** Longest matching prefix for a full 32-bit address, or null if none matches. */
    public String longestPrefixMatch(int address) {
        Node cur = root;
        String best = null;
        for (int i = 0; i < 32 && cur != null; i++) {
            if (cur.isPrefixEnd) {
                best = cur.prefixLabel;        // deepest "prefix ends here" seen so far
            }
            int bit = (address >>> (31 - i)) & 1;
            cur = cur.children[bit];
        }
        if (cur != null && cur.isPrefixEnd) {
            best = cur.prefixLabel;           // check the final node too
        }
        return best;
    }
}

```

Operation	Time	Space
Insert one prefix	$O(\text{prefixLen})$ — at most 32	$O(\text{prefixLen})$ new nodes worst case
Lookup (longest-prefix match)	$O(32)$ — one pass over the address bits	$O(1)$ extra

The trie depth is bounded by 32 (the address width), so both operations are effectively constant-time regardless of how many prefixes are stored — that bound is exactly why routers use this structure instead of scanning every prefix on every packet.

What to say

"If I saw an IP-routing or CIDR question, I'd model it as longest-prefix match: insert each prefix into a binary trie keyed on its leading bits, then walk the trie along the target address's bits, remembering the deepest 'prefix ends here' marker I pass through. I want to be upfront

that this is a conceptual pattern I'm including for its own elegance, not a question I've seen reported for any specific company — I wouldn't claim otherwise in an interview."
